

파이썬 함수 탐색

예상 소요 시간: 15분

목표:

이 읽기를 마치면 다음을 할 수 있어야 합니다:

- 1. 함수 개념과 프로그래밍에서 함수의 중요성을 설명할 수 있다.
- 2. 입력을 받아 작업을 수행하는 함수를 작성할 수 있다.
- 3. len(), sum() 등과 같은 내장 함수를 효과적으로 사용할 수 있다.
- 4. Python에서 자신의 함수를 정의하고 사용할 수 있다.
- 5. 전역 변수와 지역 변수의 범위를 구분할 수 있다.
- 6. 함수 내에서 루프를 사용할 수 있다.
- 7. 함수를 사용하여 데이터 구조를 수정할 수 있다.

함수 소개

함수는 특정 작업이나 계산을 캡슐화하는 기본 구성 요소입니다. 수학에서 함수가 입력을 받아 출력을 생성하는 것처럼, 프로그래밍 함수도 유사하게 작동합니다. 함수는 입력을 받고, 미리 정의된 작업이나 계산을 실행한 다음 출력을 반환합니다.

함수의 목적

함수는 코드의 모듈화와 재사용성을 촉진합니다. 프로그램 내에서 여러 번 수행해야 하는 작업이 있다고 가정해 보세요. 동일한 코드를 여러 곳에 복제하는 대신, 함수를 한 번 정의하고 필요할 때마다 호출할 수 있습니다. 이렇게 하면 중복이 줄어들고 코드 관리 및 유지보수가 쉬워집니다.

함수 사용의 이점

- 모듈화:** 함수는 복잡한 작업을 관리 가능한 구성 요소로 나눕니다.
- 재사용성:** 함수는 코드를 다시 작성하지 않고 여러 번 사용할 수 있습니다.
- 가독성:** 의미 있는 이름을 가진 함수는 코드 이해를 향상시킵니다.
- 디버깅:** 함수를 격리하면 문제 해결과 수정이 용이해집니다.
- 추상화:** 함수는 사용자 친화적인 인터페이스 뒤의 복잡한 프로세스를 단순화합니다.
- 협업:** 팀원들이 서로 다른 함수에서 동시에 작업할 수 있습니다.
- 유지보수:** 함수에서 이루어진 변경 사항은 사용되는 모든 곳에 자동으로 적용됩니다.

함수가 입력을 받고, 작업을 수행하며, 출력을 생성하는 방법

입력 (매개변수)

함수는 데이터에서 작동하며, 데이터는 입력으로 받을 수 있습니다. 이러한 입력을 *매개변수* 또는 *인수*라고 합니다. 매개변수는 함수가 작업을 수행하는 데 필요한 정보를 제공합니다. 매개변수를 함수에 전달하는 값으로 생각하면, 함수가 특정 데이터로 작업할 수 있게 됩니다.

작업 수행

함수가 입력(매개변수)을 받으면 미리 정의된 작업이나 계산을 실행합니다. 이러한 작업에는 계산, 데이터에 대한 작업 또는 더 복잡한 작업이 포함될 수 있습니다. 함수의 목적이 수행하는 작업을 결정합니다. 예를 들어, 함수는 숫자의 합을 계산하거나, 목록을 정렬하거나, 텍스트를 포맷하거나, 데이터베이스에서 데이터를 가져올 수 있습니다.

출력 생성

작업을 수행한 후, 함수는 출력을 생성할 수 있습니다. 이 출력은 함수 내에서 수행된 작업의 결과입니다. 함수가 호출한 코드에 “반환”하는 값입니다. 출력을 함수 작업의 최종 산물로 생각하세요. 이 출력을 코드에서 사용하거나, 변수에 할당하거나, 다른 함수에 전달하거나, 화면에 출력할 수도 있습니다.

예시:

calculate_total이라는 이름의 함수가 두 개의 숫자를 입력(매개변수)으로 받아들이고, 이를 더한 후 합계를 출력으로 생성한다고 가정해 보겠습니다. 작동 방식은 다음과 같습니다:

```
def calculate_total(a, b): # Parameters: a and b
    total = a + b          # Task: Addition
    return total           # Output: Sum of a and b
result = calculate_total(5, 7) # Calling the function with inputs 5 and 7
print(result)              # Output: 12
```

파이썬의 내장 함수

파이썬은 다양한 기능을 제공하는 풍부한 내장 함수 세트를 가지고 있습니다. 이러한 함수는 사용하기에 용이하며, 내부 구현 방식에 대해 걱정할 필요가 없습니다. 대신 각 함수가 무엇을 하는지와 효과적으로 사용하는 방법을 이해하는 데 집중할 수 있습니다.

내장 함수 또는 미리 정의된 함수 사용하기

내장 함수를 사용하려면 함수 이름을 괄호와 함께 호출하면 됩니다. 필요한 인수나 매개변수는 이 괄호 안에 전달됩니다. 그러면 함수는 미리 정의된 작업을 수행하고 코드에서 사용할 수 있는 출력을 반환할 수 있습니다.

다음은 일반적으로 사용되는 내장 함수의 몇 가지 예입니다:

len(): 시퀀스 또는 컬렉션의 길이를 계산합니다.

```
string_length = len("Hello, World!") # Output: 13
list_length = len([1, 2, 3, 4, 5])    # Output: 5
```

sum(): 반복 가능한 요소(리스트, 튜플 등)를 합산합니다.

```
total = sum([10, 20, 30, 40, 50]) # Output: 150
```

max(): 반복 가능한 객체에서 최대 값을 반환합니다.

```
highest = max([5, 12, 8, 23, 16]) # Output: 23
```

min(): 반복 가능한 객체에서 최소값을 반환합니다.

```
lowest = min([5, 12, 8, 23, 16]) # Output: 5
```

Python의 내장 함수는 len() 및 sum()과 같은 기본 작업부터 더 전문화된 작업에 이르기까지 다양한 기능을 제공합니다.

함수 정의하기

함수를 정의하는 것은 당신의 미니 프로그램을 만드는 것과 같습니다:

1. def 다음에 함수 이름과 괄호를 사용하세요.

함수를 정의하는 구문은 다음과 같습니다:

```
def function_name():
    pass
```

A "pass" 문장은 프로그래밍 함수에서 자리 표시자 또는 아무 작업도 하지 않는(no-op) 문장입니다. 기능이나 구현을 지금 당장 명시하고 싶지 않을 때, 문법적으로 함수나 코드 블록을 정의하고 싶을 때 사용합니다.

- **자리 표시자:** "pass"는 함수나 코드 블록 내에 나중에 작성할 코드를 위한 임시 자리 표시자로 작용합니다.
- **문법 요구 사항:** Python과 같은 많은 프로그래밍 언어에서는 함수나 조건 블록을 정의할 때 "pass"를 사용하는 것이 필요합니다. 이는 코드가 아무 작업도 하지 않더라도 문법적으로 올바르게 유지되도록 합니다.
- **아무 작업도 하지 않음:** "pass" 자체는 의미 있는 작업을 수행하지 않습니다. 인터프리터가 "pass"를 만나면, 코드를 실행하지 않고 단순히 다음 문장으로 넘어갑니다.

함수 매개변수:

- 매개변수는 함수의 입력과 같습니다.
- 함수 정의 시 괄호 안에 들어갑니다.
- 함수는 여러 개의 매개변수를 가질 수 있습니다.

예:

```
def greet(name):  
    return "Hello, " + name  
result = greet("Alice")  
print(result) # Output: Hello, Alice
```

Docstrings (문서 문자열)

- 문서 문자열은 함수가 수행하는 작업을 설명합니다.
- 함수 정의 아래의 세 개의 따옴표 안에 위치합니다.
- 다른 개발자가 귀하의 함수를 이해하는 데 도움이 됩니다.

예:

```
def multiply(a, b):  
    """  
    This function multiplies two numbers.  
    Input: a (number), b (number)  
    Output: Product of a and b  
    """  
    print(a * b)  
multiply(2,6)
```

반환문

- 반환은 함수에서 값을 돌려줍니다.
- 함수의 실행을 종료하고 결과를 보냅니다.
- 함수는 다양한 유형의 데이터를 반환할 수 있습니다.

예:

```
def add(a, b):  
    return a + b  
sum_result = add(3, 5) # sum_result gets the value 8
```

범위와 변수 이해하기

범위는 변수를 볼 수 있고 사용할 수 있는 곳입니다:

- **전역 범위:** 함수 외부에서 정의된 변수; 어디에서나 접근 가능
- **지역 범위:** 함수 내부의 변수; 해당 함수 내에서만 사용 가능

예시:

1부: 전역 변수 선언

```
global_variable = "I'm global"
```

이 줄은 `global_variable`이라는 전역 변수를 초기화하고 그 값으로 “I’m global”을 할당합니다.

전역 변수는 프로그램 전체에서, 함수 내부와 외부 모두에서 접근 가능합니다.

2부: 함수 정의

```
def example_function():  
    local_variable = "I'm local"  
    print(global_variable) # Accessing global variable  
    print(local_variable)  # Accessing local variable
```

여기에서 `example_function()`이라는 함수를 정의합니다.

이 함수 내에서:

- `local_variable`이라는 지역 변수가 선언되고 문자열 값 “I’m local.”로 초기화됩니다. 이 변수는 함수에 국한되어 있으며 함수의 범위 내에서만 접근할 수 있습니다.
- 그런 다음 함수는 **전역 변수 (`global_variable`)와 지역 변수 (`local_variable`)**의 값을 출력합니다. 이는 함수 내에서 전역 변수와 지역 변수에 접근할 수 있음을 보여줍니다.

3부: 함수 호출

```
example_function()
```

이 부분에서는 `example_function()`을 호출하여 실행합니다. 이로 인해 함수의 코드가 실행됩니다. 이 함수 호출의 결과로, 함수 내의 전역 변수와 지역 변수의 값이 출력됩니다.

4부: 함수 외부에서 전역 변수 접근하기

```
print(global_variable) # Accessible outside the function
```

함수를 호출한 후, 함수 외부에서 전역 변수 `global_variable`의 값을 출력합니다. 이는 전역 변수가 함수 내부와 외부에서 모두 접근 가능하다는 것을 보여줍니다.

5부: 함수 외부에서 지역 변수에 접근 시도

```
# print(local_variable) # Error, local variable not visible here
```

이 부분에서는 함수 외부에서 지역 변수 `local_variable`의 값을 출력하려고 하고 있습니다. 그러나 이 줄은 오류를 발생시킬 것입니다.

지역 변수는 정의된 함수의 범위 내에서만 보이고 접근할 수 있습니다.

그 범위 외부에서 접근하려고 하면 `NameError`가 발생합니다.

루프와 함께 함수 사용하기

함수와 루프의 결합

1. 함수는 루프가 포함된 코드를 가질 수 있습니다.
2. 이는 복잡한 작업을 더 조직적으로 만듭니다.
3. 루프 코드는 반복 가능한 함수가 됩니다.

예:

```
def print_numbers(limit):
    for i in range(1, limit+1):
        print(i)
print_numbers(5) # Output: 1 2 3 4 5
```

코드 조직 및 재사용성 향상

1. 함수는 유사한 작업을 그룹화하여 이해하기 쉽게 만듭니다.
2. 함수 내에서 루핑은 코드를 깔끔하게 유지합니다.
3. 작업을 반복하기 위해 함수를 재사용할 수 있습니다.

예시

```
def greet(name):
    return "Hello, " + name
for _ in range(3):
    print(greet("Alice"))
```

함수로 데이터 구조 수정하기

이 예시에서는 Python과 리스트를 데이터 구조로 사용합니다. 이 예제에서는 리스트에 요소를 추가하고 제거하는 함수를 생성할 것입니다.

1부: 빈 리스트 초기화

```
# Define an empty list as the initial data structure
my_list = []
```

이 부분에서는 `my_list`라는 이름의 빈 리스트를 생성합니다. 이 빈 리스트는 코드 전반에 걸쳐 수정할 데이터 구조로 사용됩니다.

2부: 요소를 추가하는 함수를 정의하기

```
# Function to add an element to the list
def add_element(data_structure, element):
```

```
data_structure.append(element)
```

여기서 `add_element` 라는 함수를 정의합니다. 이 함수는 두 개의 매개변수를 받습니다:

- `data_structure`: 이 매개변수는 요소를 추가하고자 하는 리스트를 나타냅니다.
- `element`: 이 매개변수는 리스트에 추가하고자 하는 요소를 나타냅니다.

함수 내부에서는 `append` 메서드를 사용하여 제공된 요소를 `data_structure`에 추가합니다. 이는 리스트로 가정됩니다.

3부: 요소를 제거하는 함수를 정의하기

```
# Function to remove an element from the list
def remove_element(data_structure, element):
    if element in data_structure:
        data_structure.remove(element)
    else:
        print(f"{element} not found in the list.")
```

이 부분에서는 `remove_element` 라는 또 다른 함수를 정의합니다. 이 함수는 두 개의 매개변수를 받습니다:

- `data_structure`: 요소를 제거하고자 하는 목록
- `element`: 목록에서 제거하고자 하는 요소

함수 내부에서는 조건문을 사용하여 요소가 `data_structure`에 존재하는지 확인합니다. 존재한다면, `remove` 메서드를 사용하여 요소의 첫 번째 발생을 제거합니다. 요소를 찾을 수 없는 경우, 목록에서 요소를 찾을 수 없다는 메시지를 출력합니다.

4부: 목록에 요소 추가

```
# Add elements to the list using the add_element function
add_element(my_list, 42)
add_element(my_list, 17)
add_element(my_list, 99)
```

여기에서 `add_element` 함수를 사용하여 세 개의 요소(42, 17, 99)를 `my_list`에 추가합니다. 이러한 요소는 함수 호출을 통해 한 번에 하나씩 추가됩니다.

5부: 현재 목록 인쇄

```
# Print the current list
print("Current list:", my_list)
```

이 부분은 현재 `my_list`의 상태를 콘솔에 출력하여 지금까지 추가된 요소들을 볼 수 있게 해줍니다.

6부: 목록에서 요소 제거하기

```
# Remove an element from the list using the remove_element function
remove_element(my_list, 17)
remove_element(my_list, 55) # This will print a message since 55 is not in the list
```

이 부분에서는 `remove_element` 함수를 사용하여 `my_list`에서 요소를 제거합니다. 먼저, 17을 제거하려고 시도하는데 (리스트에 존재하는 요소입니다), 그 다음에는 55를 제거하려고 시도합니다 (리스트에 없는 요소입니다). `remove_element`에 대한 두 번째 호출은 55를 찾을 수 없다는 메시지를 출력할 것입니다.

7부: 업데이트된 목록 인쇄

```
# Print the updated list
print("Updated list:", my_list)
```

마지막으로, 업데이트된 `my_list`를 콘솔에 출력합니다. 이를 통해 정의된 함수를 사용하여 요소를 추가하고 제거함으로써 리스트에 대한 수정 사항을 관찰할 수 있습니다.

결론

축하합니다! 당신은 Python 함수에 대한 읽기 교육 실습을 완료했습니다. 함수의 중요성과 이를 효과적으로 생성하고 사용하는 방법에 대한 확고한 이해를 얻었습니다. 이러한 기술은 Python 프로젝트에서 더 조직적이고 모듈화된 강력한 코드를 작성하는 데 도움이 될 것입니다.



Skills Network